# TABLE OF CONTENTS

SECTION 1

# Project Overview

This report documents the end-to-end design, implementation, and evaluation of an intelligent car auction system developed by Ayush Raigandhi. The project combines supervised machine learning — specifically gradient-boosted trees — with a rule-based adaptive bidding agent to compete in live vehicle auctions. The system estimates the fair market value of any car given its attributes, then bids strategically against rivals in real time to maximise profit while protecting a finite bankroll.

| $500k | 85% | 100 | 20% |
|---|---|---|---|
| Starting Bankroll | Margin Factor | Optuna Trials | Validation Split |

> *Key insight: By predicting log(sellingprice) rather than the raw price, the model avoids penalising large absolute errors on cheap cars and large errors on expensive cars equally — a critical design choice for auction environments where proportional accuracy matters most.*

## System Architecture

The project has two tightly coupled components. The **analysis notebook** (analysis_AyushRaigandhi.ipynb) handles the offline pipeline: raw data ingestion, cleaning, EDA, feature engineering, model training via CatBoost, and artefact serialisation. The **bidding agent** (agent_AyushRaigandhi.py) loads those serialised artefacts at run-time, calls *analyze_item()* to price a car, then repeatedly calls *place_bid()* as the auction unfolds.

| Component | File | Responsibility |
|---|---|---|
| Analysis Pipeline | `analysis_AyushRaigandhi.ipynb` | EDA, feature engineering, model training, artefact export |
| Prediction Engine | `model_AyushRaigandhi.pkl` | CatBoost regressor — predicts log(sellingprice) |
| Mapping Store | `mappings_AyushRaigandhi.pkl` | State/brand averages, popularity counts, feature column order |
| Auction Agent | `agent_AyushRaigandhi.py` | LiveAuctionAgent: feature engineering, valuation, bidding strategy |

SECTION 2

# Dataset & Problem Statement

The raw training data is sourced from *car_auction_train.csv*, a tabular dataset representing historical vehicle auction records in the United States. Each row corresponds to a single vehicle transaction and carries the following feature groups:

| Group | Features | Type |
|---|---|---|
| Categorical | make, model, trim | Brand identity & configuration |
| Structural | year, body, transmission | Physical vehicle class |
| Operational | state, condition, odometer | Usage history & geography |
| Aesthetic | color, interior | Cosmetic attributes |
| Target | sellingprice | Continuous — auction hammer price ($) |

## Problem Statement

Given a set of vehicle attributes, predict the auction hammer price as accurately as possible, then use that prediction to drive real-time bidding decisions that win cars at a discount to their estimated value. This is a supervised regression problem with a downstream decision-making component.

> *The model predicts log(1 + sellingprice) and the agent calls expm1() to recover the dollar value. This log-transform is not cosmetic — it normalises the heavily right-skewed price distribution and makes RMSE interpretable in proportional terms.*

SECTION 3

# Data Cleaning Pipeline

Raw auction data carries significant noise: missing values, miscoded strings, and outliers in both odometer readings and selling prices. A rigorous cleaning pipeline was applied before any modelling work.

## 3.1 Handling Missing Values

**Categorical columns** — All string and categorical columns are imputed with the literal token *'unknown'*, then lower-cased and stripped of whitespace. This preserves the column's cardinality without introducing a spurious new category that could skew frequency-based encodings.

**Numerical columns** — *condition* and *odometer* are imputed with their respective training-set medians. The median is robust to the right-skewed distribution of both columns; mean imputation would inflate central tendency estimates.

## 3.2 Outlier Detection & Clipping

| | | |
|---|---|---|
| condition | Hard filter: keep [0.0, 5.0] | Auction grading scale is bounded by definition |
| odometer | Log-transform then IQR clip [Q1-1.5×IQR, Q3+1.5×IQR] | Log reduces skew; IQR removes extreme mileage outliers |
| sellingprice | Log-transform then IQR clip on log scale | Right-skewed; log-space clipping is more principled than raw clipping |

> *A key design decision: odometer is dropped from the final feature set after odometer_log is created. Keeping both would introduce multicollinearity and allow the model to re-learn the raw scale, partially defeating the normalisation.*

SECTION 4

# Exploratory Data Analysis

Three primary analytical questions guided the EDA phase, each directly influencing a downstream feature engineering or modelling decision.

1 **Depreciation curves by body type**

A faceted lmplot with LOWESS smoothers was used to visualise how median log-price decays with vehicle age for the five most common body types (Sedan, SUV, Pickup, Coupe, Van). The non-linear depreciation curves confirmed that a simple linear age term is insufficient — motivating the age_bucket categorical feature and the years_until_floor numeric feature.

2 **Condition vs. price distribution**

A violin plot of sellingprice_log across integer condition ratings revealed that the relationship is monotonically increasing but non-linear: the jump from condition 4 to 5 is disproportionately large, while the jump from 1 to 2 is modest. This motivated the binary deprecated flag (condition <= 2.0), which explicitly signals near-worthless vehicles to the model.

3 **State-level average prices**

A sorted bar chart of mean selling price by U.S. state revealed significant regional variation — coastal states and high-income regions command price premiums. This directly motivated the state_avg_price target-encoded feature, computed exclusively on training data to prevent leakage.

4 **Usage intensity**

A scatter of usage_intensity (odometer / car_age) against log-price showed a clear negative relationship. Vehicles driven more than 25,000 miles per year — flagged as overused — cluster at the low end of the price distribution, motivating the corresponding binary feature.

5 **Correlation heatmap**

A Pearson correlation heatmap of all numeric features confirmed expected relationships: odometer_log strongly negatively correlated with price; condition positively correlated. The heatmap also revealed that miles_per_year was largely redundant with odometer_log and usage_intensity, leading to its exclusion from the final feature set.

SECTION 5

# Feature Engineering

Feature engineering is the most consequential step in this pipeline. A total of 15 derived features were created across six conceptual categories, each grounded in domain knowledge of vehicle depreciation and auction market dynamics.

## 5.1 Temporal & Usage Features

**car_age** — Computed as 2026 minus the model year. Clamped to a minimum of 1 to avoid division-by-zero in downstream features. This is the most fundamental continuous driver of depreciation.

**odometer_log** — Natural log of (1 + odometer). Reduces heavy right skew and compresses extreme values while preserving monotonicity.

**usage_intensity** — Raw odometer divided by car_age. Captures whether a car has been driven harder than average for its age, independent of both age and mileage alone.

**overused** — Boolean flag: usage_intensity > 25,000 miles/year. Isolates the cohort of commercially abused vehicles that depreciate non-linearly.

## 5.2 Lifecycle Bucket Features

**age_bucket** — Ordinal categorical with six bins derived from observed depreciation inflection points:

| Bucket | Age Range | Market Interpretation |
|---|---|---|
| new | 0–3 years | Near-new, minimal depreciation hit |
| midsized | 4–7 years | Steepest depreciation phase |
| old | 8–12 years | Slowing depreciation, reliability concerns |
| very_old | 13–20 years | Near floor price |
| floor | 21–25 years | At depreciation floor |
| vintage | 25+ years | Potential appreciation; collector market |

**years_until_floor** — max(0, 25 - car_age). Provides the model a continuous signal of how far the car is from reaching its residual floor value.

**is_vintage** — Boolean: car_age >= 30. Flags the collector segment where normal depreciation curves reverse.

**deprecated** — Boolean: condition <= 2.0. Marks near-end-of-life vehicles that price far below their age cohort.

## 5.3 Brand Prestige — Premium Score

A manually curated integer score from 2 (Daewoo/Geo) to 10 (Rolls-Royce/Ferrari) encodes brand prestige. This gives the model a single, interpretable numeric proxy for brand-level price premium without high-cardinality one-hot encoding. Brands absent from the lookup default to 5 (mid-market).

## 5.4 Vehicle Segment Flags

| | | |
|---|---|---|
| is_suv | `suv\|explorer\|tahoe\|suburban\|x5\|q5\|q7\|rav4\|cr-v` | Sport Utility Vehicle |
| is_sports | `m3\|m5\|amg\|rs\|corvette\|mustang\|challenger\|911` | Performance / Sports Car |
| is_hybrid_ev | `hybrid\|electric\|ev\|plug-in` | Electrified Powertrain |

## 5.5 Target-Encoded Geographic & Brand Features

**state_avg_price** and **brand_avg_price** are computed from training data only and mapped onto both train and validation sets. Computing these mappings after the train/validation split prevents data leakage — a common pitfall when using target encoding. Unseen states or brands fall back to the global training mean.

**model_popularity** counts how often each model string appears in the training corpus. Rare models are implicitly penalised because the model cannot learn a reliable price signal from sparse observations. **rare_make** is a binary flag for makes with fewer than 10 appearances, giving the model an explicit signal for data-sparse brands.

SECTION 6

# Model Training & Hyperparameter Optimisation

## 6.1 Model Selection — CatBoost

CatBoostRegressor was selected as the primary model for several principled reasons:

- Native categorical feature support eliminates the need for manual one-hot or ordinal encoding of high-cardinality columns such as make, model, trim, and body.
- Ordered boosting reduces overfitting on the training split by preventing target leakage within the boosting process — particularly valuable when state_avg_price and brand_avg_price already carry partial target information.
- Built-in early stopping prevents overfitting without the need for a separate held-out pruning set, keeping the full 80% training split available for learning.
- Competitive benchmark performance on tabular regression tasks, often matching or exceeding XGBoost and LightGBM with less preprocessing required.

## 6.2 Hyperparameter Search — Optuna

Optuna's Tree-structured Parzen Estimator (TPE) sampler was used instead of RandomizedSearchCV to conduct 100 trials of Bayesian hyperparameter optimisation. TPE learns from previous trials and concentrates sampling in promising regions of the search space — far more efficient than random search at this trial count.

| Hyperparameter | Search Space | Rationale |
|---|---|---|
| iterations | Categorical: {500, 1000, 1500} | Controls tree count; more trees = higher capacity but slower inference |
| learning_rate | Log-uniform: [0.01, 0.1] | Log-uniform prior reflects that small rates are often better |
| depth | Integer: [4, 8] | Shallower trees reduce overfitting; deeper trees capture interactions |
| l2_leaf_reg | Integer: {1, 3, 5, 7, 9} | L2 regularisation on leaf values; higher = smoother, more regularised model |
| early_stopping_rounds | Fixed: 50 | Stop if validation RMSE does not improve for 50 consecutive rounds |

## 6.3 Train / Validation Split

An 80/20 stratified random split (random_state=42) was used. The target-encoded mappings (state_avg_price, brand_avg_price) are computed exclusively from the training fold before being applied to both folds, preventing any form of data leakage from the validation set into the features.

SECTION 7

# Model Evaluation & Performance

The final model is evaluated on the held-out validation set. Because the model predicts log(1 + sellingprice), the primary metric is RMSE computed after applying expm1() to recover the original dollar scale.

## 7.1 Evaluation Metrics

RMSE in dollars is the headline metric because it penalises large prediction errors quadratically — aligning with the auction context where a large misvaluation leads to either overpaying significantly or missing a deal entirely.

## 7.2 Feature Importance

CatBoost's built-in feature importance (PredictionValuesChange) was extracted and plotted post-training. Based on domain knowledge and the EDA findings, the top expected drivers of importance are:

| Expected Rank | Feature | Why It Matters |
|---|---|---|
| 1–2 | brand_avg_price, state_avg_price | Target-encoded features directly encode mean price by group — strongest signal |
| 3 | odometer_log | Mileage is the single most visible usage signal to buyers |
| 4 | car_age | Primary driver of depreciation |
| 5 | condition | Direct auction-grade signal |
| 6 | premium_score | Encodes brand prestige as a compact numeric signal |
| 7+ | model, make, trim, age_bucket … | Secondary categorical signals; high cardinality but handled natively by CatBoost |

SECTION 8

# Auction Agent Architecture

The *LiveAuctionAgent* class is the runtime component of the system. It loads serialised artefacts at initialisation, prices cars on demand, and executes a multi-signal adaptive bidding strategy in real time. All state is held in instance variables, making the class safe for concurrent use across multiple simultaneous auctions by instantiating separate objects.

## 8.1 Initialisation

The constructor loads two joblib artefacts: *model_AyushRaigandhi.pkl* (the trained CatBoost regressor) and *mappings_AyushRaigandhi.pkl* (five lookup dictionaries). Warnings are suppressed during loading to avoid version-mismatch noise from scikit-learn's joblib interface.

| | | |
|---|---|---|
| bankroll | float, $500,000 | Total capital available across all auctions in a session |
| predicted_value | float | Current car's estimated fair value; reset by analyze_item() |
| round_number | int | Bid count for the current auction; drives geometric decay |
| last_own_bid | float | Agent's most recent bid; used to detect rival counter-bids |
| last_rival_bid | float | Tracks rival's last known bid position |
| rival_jumps | list[float] | History of rival increment sizes; used to compute aggression score |

## 8.2 Feature Engineering at Inference

The private method *_build_features()* replicates the notebook's entire feature engineering pipeline in pure Python. It accepts a raw feature dictionary and returns the complete derived feature set. This design ensures that inference is 100% consistent with training — no feature drift, no schema mismatch. The returned dictionary is aligned to the saved feature_columns list before passing to the model, guaranteeing correct column ordering.

## 8.3 analyze_item() — Valuation

Accepts a raw feature dictionary, calls *_build_features()*, constructs a single-row DataFrame aligned to the training columns, calls model.predict(), and applies expm1() to recover the dollar estimate. All per-auction state (round_number, last_own_bid, rival_jumps) is reset, making the agent ready for a fresh auction.

## 8.4 auction_result() — Bankroll Management

Called after each auction concludes. If the agent won, the winning_bid is subtracted from the bankroll (clamped to zero). Loss events incur no cost. The actual_price parameter is available for post-hoc analysis but is not

currently used to update the model — the system is offline-trained only.

SECTION 9

# Bidding Strategy Deep-Dive

The bidding strategy in *place_bid()* is the most algorithmically rich component of the system. It synthesises three independent market signals each round into a single adaptive bid increment. The key architectural insight is that no single signal is sufficient — all three must be combined to handle the full range of auction dynamics.

## 9.1 Constants & Their Rationale

| Constant | Value | Role |
|---|---|---|
| MARGIN_FACTOR | 0.85 | Hard ceiling: never bid above 85% of predicted value. Ensures a minimum 15% profit margin on any won lot. |
| BANKROLL_CAP | 0.15 | Never expose more than 15% of current bankroll on a single car. Prevents catastrophic loss from model misjudgement. |
| BASE_AGGRESSION | 0.30 | Fraction of headroom to bid on round 1 before decay and signal modulation. |
| DECAY_RATE | 0.88 | Geometric decay factor per round. After 5 rounds, aggression is 0.88^4 ≈ 60% of round-1 aggression. |
| MIN_INCREMENT | $100 | Floor bid increment. Ensures participation never falls below a token raise. |

## 9.2 Signal 1 — Deal Quality

Deal quality measures how large the gap between predicted value and current bid is, expressed as a fraction of predicted value:

$$\texttt{deal\_quality} = (\texttt{predicted\_value} - \texttt{current\_bid}) / \texttt{predicted\_value} \in [0, 1]$$

A deal_quality of 0.9 means the car is currently priced at only 10% of its predicted value — an extraordinary opportunity warranting aggressive bidding. A deal_quality near 0 means the current price is close to the ceiling; the agent should back off. This signal enters the formula as (0.5 + deal_quality), ensuring it never fully kills the increment even at high prices.

## 9.3 Signal 2 — Rival Aggression

After each round where a rival outbid the agent, the rival's increment (current_bid – last_own_bid) is appended to rival_jumps. The running mean of rival_jumps, normalised by predicted value, gives rival_aggression:

$$\texttt{rival\_factor} = 1.0 - \texttt{clip}(\texttt{rival\_aggression} \times 8,\ 0.0,\ 0.40) \in [0.60,\ 1.0]$$

A rival making large jumps (e.g. 10% of predicted value per round) is classified as highly confident. The agent reduces its own increment by up to 40%, preserving capital rather than fighting a well-informed competitor. A timid rival (small jumps) triggers no reduction — one assertive raise can close the auction.

## 9.4 Signal 3 — Position Factor (Bid-to-Value Ratio)

| Bid / Predicted Value | position_factor | Interpretation |
|---|---|---|
| < 0.40 | 1.35 | Early stage — strong discount; bid boldly to establish position |
| 0.40 – 0.60 | 1.10 | Mid-range — still room to close; mildly elevated aggression |
| 0.60 – 0.75 | 0.90 | Approaching ceiling — begin pulling back |
| > 0.75 | 0.65 | Near ceiling — minimum increments only; preserve margin |

## 9.5 Combining the Three Signals

The three signals are combined into a single increment formula:

```
base_decay = BASE_AGGRESSION × DECAY_RATE^(round − 1) context_k = base_decay × rival_factor × position_factor × (0.5 + deal_quality) increment = max(MIN_INCREMENT, headroom × context_k) next_bid = min(current_bid + increment, ceiling, bankroll)
```

The result is hard-capped at three independent ceilings: the value ceiling (85% of predicted), the bankroll ceiling (15% of remaining bankroll), and the current bankroll itself. The first ceiling that binds takes effect, ensuring no single auction can exceed any safety constraint.

SECTION 10

# Artefacts & Deployment

| Artefact | Format | Contents |
|---|---|---|
| model_AyushRaigandhi.pkl | joblib pickle | Trained CatBoostRegressor with best Optuna hyperparameters |
| mappings_AyushRaigandhi.pkl | joblib pickle | Dict with state_avg_mapping, brand_avg_mapping, model_popularity, make_freq, global_mean, feature_columns |
| agent_AyushRaigandhi.py | Python module | LiveAuctionAgent class; self-contained deployment unit |

> *The feature_columns list stored in mappings.pkl is the critical contract between training and inference. The agent's _build_features() output is sliced to exactly this column list and in this order before calling model.predict(). Any change to the notebook's feature set requires regenerating both artefacts together.*

SECTION 11

# Conclusion & Future Work

This project demonstrates a complete ML-to-deployment pipeline for a real-world auction pricing and bidding problem. The key contributions are:

- A principled data cleaning pipeline with log-transforms, median imputation, and IQR clipping — all decisions grounded in the data's distributional properties.
- Fifteen domain-informed derived features that encode vehicle depreciation dynamics, brand prestige, segment classification, and geographic market variation.
- A CatBoost model trained with Bayesian hyperparameter search (100 Optuna trials) on log-transformed prices, producing a proportionally accurate valuation engine.
- A three-signal adaptive bidding agent that balances deal quality, rival intelligence, and price position to win cars profitably within hard bankroll constraints.
- A clean deployment architecture where inference exactly mirrors training through a shared feature engineering function and a serialised column contract.

## Future Directions

- **Online learning** — Update brand_avg_price and state_avg_price incrementally as auction results arrive, allowing the model to adapt to shifting regional markets without full retraining.
- **Rival modelling** — Maintain a per-rival bid history to classify rival strategy type (value buyer, momentum bidder, shill) and adapt bidding accordingly.
- **Multi-lot bankroll allocation** — Implement a portfolio optimisation layer that allocates capital across upcoming lots based on expected profit margins, rather than the current flat 15% per-lot cap.
- **Uncertainty quantification** — Augment the CatBoost point estimate with a quantile regression model to obtain confidence intervals, allowing the agent to bid more conservatively on high-uncertainty valuations.
- **Image features** — Incorporate vehicle photos via a CNN or CLIP embedding to capture condition and cosmetic state beyond the numeric condition score.

End of Report — Ayush Raigandhi | Car Auction Intelligence System | May 29, 2026